

Plano de Desenvolvimento

Agosto → Novembro de 2026 · 6 sprints · 4 pessoas

Pet+ — Plano de Desenvolvimento (Ago → Nov 2026)

Documento-guia único para o time. Reúne o **estado atual** do projeto, as **decisões de negócio** já alinhadas, as **convenções** de código e o **roteiro por sprint** até a primeira semana de novembro de 2026.

As tasks detalhadas (uma por arquivo) estão em `/Sprints`. Cada arquivo é autossuficiente: o dev abre um `.md` e sabe o que fazer, onde, por quê e como validar.

1. O que é o Pet+

Marketplace de pets com três frentes: **loja de produtos**, **hotéis** (hospedagem) e **pet sitters** (passeio/day care/cuidado). Cinco papéis de usuário:

role_id	Papel	O que faz
1	ADMIN	Aprova parceiros, avalia pet sitters, vê usuários e métricas da plataforma
2	LOJISTA	Vende produtos na loja
3	HOTEL	Oferece hospedagem
4	PET_SITTER	Oferece serviços de cuidado
5	TUTOR	Cliente final: compra produtos e contrata serviços

Stack

- **Backend** (`backend/`): Node.js, **Express 5**, **Prisma** + PostgreSQL (Railway), JWT, **Zod**, **pino** (logs), helmet, express-rate-limit. Testes com `node:test` + supertest.
- **Frontend** (`frontend/`): **React 18**, **Vite**, **TypeScript**, **Tailwind v4**, Radix/shadcn, **TanStack Query**, react-hook-form, recharts. Testes com Vitest + Playwright.
- **Deploy**: front na **Vercel**, back no **Railway**.

2. Estado atual (agosto/2026)

Legenda:  completo ·  parcial ·  ausente ·  precisa correção.

O que já está sólido ✅ (não retrabalhar)

Autenticação (login/registro com anti-escalamento de privilégio; forgot/reset com token hashado single-use — **exceto o envio de e-mail**), Pets, Endereços, Favoritos, criação/edição de Produtos, Carrinho *server-side* (**exceto o gate de papel, ver abaixo**), Painel Admin (aprovações, avaliações de sitter, usuários, métricas), PublicProfile, Account, logging estruturado, health check, métricas RED, CI, CSP.

O núcleo transacional NÃO existe ❌ (é o foco deste plano)

- **Checkout/Pedidos:** `Order` / `OrderItem` existem no schema mas **nada os cria**. O checkout no front é fachada (`setTimeout` + `clearCart` , sem API). Logo, histórico de pedidos e receita nos dashboards são todos mock.
- **Agendamento/Reserva:** `Service` / `Appointment` (e seus repositórios) existem mas são **código morto** — nenhuma rota os expõe. Os botões "Reservar/Agendar" são `onClick={()=>{}}` .
- **Pagamento:** nenhum gateway integrado.
- **Avaliações:** todo rating/review exibido é fabricado (`5.0` , `Math.random()`).

Correções críticas conhecidas 🔧

- **Carrinho bloqueia o tutor:** `CartController.js:108,234` e `CartContext.tsx:243` usam `role_id !== 1` (role 1 é ADMIN) — o tutor (5) toma 403. → `PED-0` / `CAR-0` .
- **Vazamentos:** `password_hash` em respostas de provider; `document` (CPF/CNPJ) em rotas públicas. → `SEC-1` .
- `is_active` ignorado no login (conta desativada continua logando). → `SEC-1` .
- **Sem migrations versionadas** (deploy via `db push` , com drift). → `INF-0` .
- **Recuperação de senha não envia e-mail** (só `console.log`). → `EML-1` .
- `Provider.status` é texto livre com sinônimos (`APROVADO` / `ATIVO` / `ACTIVE`). → `STA-1` .
- **Dashboards mock sem aviso** (Overview/Schedule/Customers/TutorOrders/TutorAppointments). → `UX-0` (banner) e depois dados reais.

O relatório de auditoria completo (com todos os 🟡 / 🟢) foi discutido com a coordenação; este plano prioriza o que destrava as jornadas e a segurança.

3. Decisões de negócio (JÁ ALINHADAS — não reabrir sem combinar)

1. **Status do parceiro vira enum:** `PENDENTE` , `APROVADO` , `REJEITADO` , `EM_REVISAO` . - Os valores legados `ATIVO` / `ACTIVE` são migrados para `APROVADO` . - "**Sitter operacional**" = `status === APROVADO` e `SitterEvaluation` do sitter em `APPROVED` (o sitter tem uma etapa extra: o questionário). - `EM_REVISAO` = cadastro **reenviado após uma recusa**, aguardando reanálise.
2. **Recusa e reenvio de cadastro:** ao recusar, o admin grava um **motivo** (`rejection_reason`) e o sistema **envia e-mail** (via Resend) com um **link para a tela de correção**. A correção **exige login**. Ao reenviar, o status vai para `EM_REVISAO` .

3. **Política de pagamento por parceiro** (`payment_policy` , configurável em Configurações): - **PRESENCIAL** (**padrão**): paga na data, presencialmente. O agendamento é confirmado pelo parceiro sem pagamento online. - **PRE_PAGO** : exige pagamento antes da confirmação. O agendamento nasce **AGUARDANDO_CONFIRMACAO** ; recusa/cancelamento gera **estorno**.
4. **Unidades de cobrança**: **hotel = diária**; **sitter = visita/hora**.
5. **Quem vende produto**: **somente LOJISTA aprovado**. Hotel/sitter/tutor não criam produtos, e produto de lojista **não-aprovado não aparece na loja**.
6. **Aceite de termos** passa a ser **capturado de verdade** (bloqueia cadastro sem aceite; grava o valor real + data/versão dos termos).
7. **Pagamento em duas camadas**: primeiro **SIMULADO** (marca o pedido/agendamento como pago) — destrava todas as jornadas. **Mercado Pago sandbox** entra por cima como **OPCIONAL** (`PGT-1`), sem refazer a modelagem.
8. **Favorite.target_id** **sem FK**: mantido como está (relação polimórfica; integridade garantida na aplicação).

Máquinas de estado

- **Pedido** (`Order.status`): `AGUARDANDO_PAGAMENTO` → `PAGO` → (`ENVIADO/CONCLUIDO`), com `CANCELADO` . (ajustar nomes na modelagem de `PED-1` / `PAG-1` .)
- **Agendamento** (`Appointment.status`): `PENDENTE` → `AGUARDANDO_CONFIRMACAO` (só `PRE_PAGO`) → `CONFIRMADO` → `CONCLUIDO` , com `RECUSADO` / `CANCELADO` . A **avaliação** só é liberada após `CONCLUIDO` .

4. Convenções de código

Backend

- **Camadas**: `routes` → `controllers` → `useCases` → `repositories/prisma` . Regras de negócio ficam nos `useCases`.
- **Erros**: sempre `next(AppError.xxx(...))` — o `errorHandler` central converte para o formato `{ error: { code, message } }` e trata Zod (422) e Prisma (P2002/P2025/P2003). **Não** usar `res.status(500).json({ error: error.message })` (vaza internals e mascara 404 como 500).
- **Validação**: Zod em toda entrada. `.strict()` / `.strip()` para barrar mass-assignment.
- **Nunca** retornar `user` cru (vaza `password_hash`) — usar `select` explícito. **Nunca** expor `document` (CPF/CNPJ) em rota pública.
- **Transações**: operações compostas (ex.: criar pedido + baixar estoque) dentro de `prisma.$transaction` .
- **Logs**: `req.log` (pino, com correlation id). Nada de `console.log` em código de produção.

Frontend

- **Dados**: `lib/services/*` chamam o `httpClient` tipado; **hooks React Query** em `lib/hooks` para cache/estados. Seguir o padrão de `useProfile` / `useAdminMetrics` .
- **Estados obrigatórios** em toda tela que busca dados: **loading**, **erro**, **vazio**, **sucesso**. Loader padrão: `HamsterLoader` .

- **Sem dados fabricados** em telas de produção. Enquanto o dado real não existe, usar **banner de demonstração** (padrão de `Finance.tsx` / `Orders.tsx`).
- **Design system**: usar componentes shadcn (`Card` , `Button` , `Table` , `Select`) e tokens `var(--color-primary-*)` — evitar hex soltos (`#3699D2`) e cards manuais.

Armadilhas do projeto (gotchas)

- **Express 5**: `req.query` é **read-only** → parsear com Zod inline (`const { x } = schema.parse(req.query)`), nunca reatribuir `req.query` .
- **Prisma** `groupBy` agrupa por timestamp exato → para métricas por dia, **bucketizar em JS**.
- **Prisma.Decimal** → converter com `Number()` antes de serializar.
- `frontend/.env.local` aponta para a API do Railway (produção). Para rodar contra o back local, use `VITE_API_URL=http://localhost:3000/api/v1` .

5. Ambiente e setup

Variáveis de ambiente (backend)

Var	Obrigatória	Nota
<code>JWT_SECRET</code>	✅ (fail-fast no boot)	≥32 chars
<code>DATABASE_URL</code>	✅	Postgres (Railway)
<code>RESEND_API_KEY</code>	a partir de <code>EML-1</code>	envio de e-mail
<code>FRONTEND_URL</code>	a partir de <code>EML-1</code>	montar links de e-mail
<code>PORT</code> / <code>NODE_ENV</code> / <code>LOG_LEVEL</code>	opcionais	têm default

`INF-0` cria o `.env.example` oficial. **Nunca** commitar `.env` real. Os segredos de produção atuais devem ser **rotacionados** (ver `INF-0`).

Rodar

```
# backend
cd backend && npm install && npx prisma generate && npm run dev
# frontend
cd frontend && npm install && npm run dev
# testes
cd backend && npm test
cd frontend && npx vitest run && npx playwright test
```

6. Fluxo de trabalho do time

- **Branches**: uma branch por task, nomeada pelo ID — ex.: `feat/PED-1-post-orders` .

- **Pull Request:** toda entrega passa por PR com **revisão de outra pessoa**. Ninguém commita direto na `main`.
- **Definition of Done** (por task): critérios de aceite atendidos + testes passando + lint/typecheck limpos + PR revisado e mergeado. Cada arquivo de task traz o DoD específico.
- **Board:** acompanhar no Trello/Notion (CSV importável entregue à coordenação). Mover o cartão por *A Fazer* → *Fazendo* → *Revisão* → *Concluído*.

7. Time e trilhas

	Trilha	Foco
R1	Backend · Comércio	Pedidos, checkout, pagamento, carrinho, estoque, pagamento do agendamento, infra
R2	Backend · Plataforma	Status de parceiro, serviços, recusa/reenvio, e-mail (Resend), avaliações, segurança, dashboards
R3	Frontend · Transacional	Checkout, pedidos, agendamento, avaliações (telas do cliente)
R4	Frontend · Painéis & QA	Configurações, dashboards, telas do parceiro/admin, testes E2E/integração

1 task principal por pessoa por semana. O backend entrega o contrato numa semana e o frontend consome na seguinte — respeitar a coluna "Depende de" de cada task.

8. Roteiro por sprint

11 semanas, de **25/ago** a **08/nov/2026**. Detalhe de cada task em [/Sprints](#).

Sprint 0 — Fundação · Semana 1 (25–31/ago) · Marco M0

Destrava o carrinho, versiona o banco, rotaciona segredos e torna honestos os dashboards mock. `PED-0` · `INF-0` · `CAR-0` · `UX-0`

Sprint 1 — Pedidos & Segurança · Semanas 2–3 (01–14/set) · Marco M1

Pedidos no backend; vazamentos fechados; `status` vira enum. `PED-1` · `SEC-1` · `CHK-1` · `CFG-1` · `PED-2` · `STA-1` · `PRD-1` · `QA-1`

Sprint 2 — Checkout & E-mail/Recusa · Semanas 4–5 (15–28/set) · Marco M2

Compra ponta a ponta (pagamento simulado); e-mail real (Resend); recusa/reenvio de cadastro. `PAG-1` · `EML-1` · `CHK-2` · `UX-2` · `AGD-1` · `REC-1` · `PED-F` · `REC-2`

Sprint 3 — Serviços & Agendamento (back) · Semanas 6–7 (29/set–12/out) · Marco M3

Serviços e agendamento no backend; produto só para lojista aprovado. `AGD-2` · `SRV-1` · `CHK-3` · `SRV-F1` · `PGT-1` ★ · `PRD-2` · `SRV-Fpub` · `SRV-F2`

Sprint 4 — Agendamento (front) & Avaliações (back) · Semanas 8–9 (13–26/out) · Marco M4

Agendamento ponta a ponta com política de pagamento; avaliações no backend. **PAG-2** · **AVL-1** · **BKG-1** · **SCH-1** · **STO-1** ★ · **AVL-2** · **BKG-2** · **SCH-2**

Sprint 5 — Fechamento · Semanas 10–11 (27/out–08/nov) · Marco M5

Avaliações no front; dashboards com dado real; segurança/infra; QA de release. **INF-2** · **DSH-1** · **AVL-F1** · **QA-2** · **SEC-2** · **DSH-2** · **AVL-F2** · **QA-3**

★ = **opcional** (pode migrar para o próximo semestre sem afetar o essencial).

Marcos

Marco	Data	Entrega verificável
M0	31/ago	Carrinho funciona p/ tutor · banco versionado · segredos rotacionados · dashboards com aviso
M1	14/set	Pedidos no backend · vazamentos fechados · status enum
M2	28/set	Compra ponta a ponta (pgto simulado) · e-mail real · recusa/reenvio
M3	12/out	Serviços + agendamento no backend · gate de produto
M4	26/out	Agendamento ponta a ponta com política de pagamento
M5	08/nov	Avaliações reais · dashboards reais · testes E2E · pronto p/ entrega

9. Backlog opcional (candidato ao próximo semestre)

Sem quebrar o essencial: **Mercado Pago real** (**PGT-1** , mantém o simulado), opção **SINAL** de pagamento, **storage de mídia** (**STO-1**) se estourar prazo, mapas/geolocalização, SMS/push/notificações, soft-delete, tela **Customers** (hoje órfã — decidir remover), limpeza de dependências (**depcheck**), unificação total do design system.

10. Riscos e recomendações

- **Semanas mais pesadas:** Sprint 2 (checkout ligado + Resend + recusa) e Sprint 4 (agendamento front + avaliações back). Se houver atraso, será nelas.
- **AGD-2** e **SRV-1** (Sprint 3) são os maiores itens de backend; se um escorregar, o agendamento no front (Sprint 4) sofre. Acompanhar de perto o **M3 (12/out)**.
- **Não acumular duas trilhas na mesma pessoa** — o encadeamento back→front depende de cada trilha entregar no prazo.
- **Cortar pelos opcionais primeiro** (★) se o cronograma apertar; as jornadas continuam íntegras.